

Stack Tecnológico

Stack de Tecnologías (The Stack)

Para garantizar una carga en **menos de 2 segundos** y soportar **10,000 activos**, utilizaremos tecnologías modernas y eficientes:

Frontend: React.js (Single Page Application - SPA).

- **Vite:** Como herramienta de construcción (build tool) para asegurar rapidez en desarrollo y producción.
- **Tailwind CSS:** Para un diseño **Mobile-First** ágil y accesible.

Backend: Node.js + Express.js.

ORM: Prisma (Actúa como puente entre el código Node.js y PostgreSQL).

Base de Datos: PostgreSQL + Supabase.

- *Justificación:* Necesitamos integridad referencial para la trazabilidad total de los activos y relaciones complejas entre usuarios, préstamos e instituciones.

Control de Versiones: Git (Uso obligatorio de ramas y commits claros).

Arquitecturas

Single Page Application (SPA) con capacidades PWA

Nuestra aplicación cargará una sola página y actualizará el contenido dinámicamente.

1. ¿Qué es una SPA (Single Page Application)?

- **Web Tradicional:** Cada vez que haces clic, la pantalla se pone en blanco y vuelve a cargar todo desde cero (como salir de una habitación y volver a entrar). Lento y feo.
- **SPA (Nosotros):** Es como una **App de celular**. Entrás una vez y nunca te vas. Si haces clic en "Perfil", solo cambia el centro de la pantalla, pero el menú y el logo se quedan quietos. **Es instantáneo.**

Regla para el Equipo (React):

- **PROHIBIDO** usar la etiqueta `<a href="...">` para navegar dentro de la app. Eso recarga la página y mata la magia.
- **SIEMPRE** usen el componente `<Link to="...">` de React Router.

2. ¿Qué es una PWA (Progressive Web App)?

Imagina esto: Es una página web que **se cree App nativa**.

- Se puede **instalar** en el celular (aparece el icono junto a Instagram o WhatsApp).
- Funciona **sin internet** (o con el Wi-Fi lento de la bodega), porque guarda datos en el caché del celular.
- Se ve en **pantalla completa** (sin la barra de direcciones de Chrome/Safari).

Regla para el Equipo (Vite + Tailwind):

- **Diseñen Mobile-First:** Primero que se vea bien en el celular (usando clases de Tailwind como `w-full`, `p-4`), luego lo ajustamos para PC (`md:w-1/2`).
- **Iconos:** Necesitamos el logo de ZF en alta calidad para que sea el icono de la app en el celular del juez.

Resumen:

- **React:** Hace que la navegación sea instantánea (**SPA**).
- **Vite:** Empaqueta todo para que el celular lo reconozca como App (**PWA**).
- **Tailwind:** Hace que los botones sean grandes y fáciles de tocar con el dedo (**Mobile First**).

API RESTful

El backend funcionará como un servidor de recursos independiente.

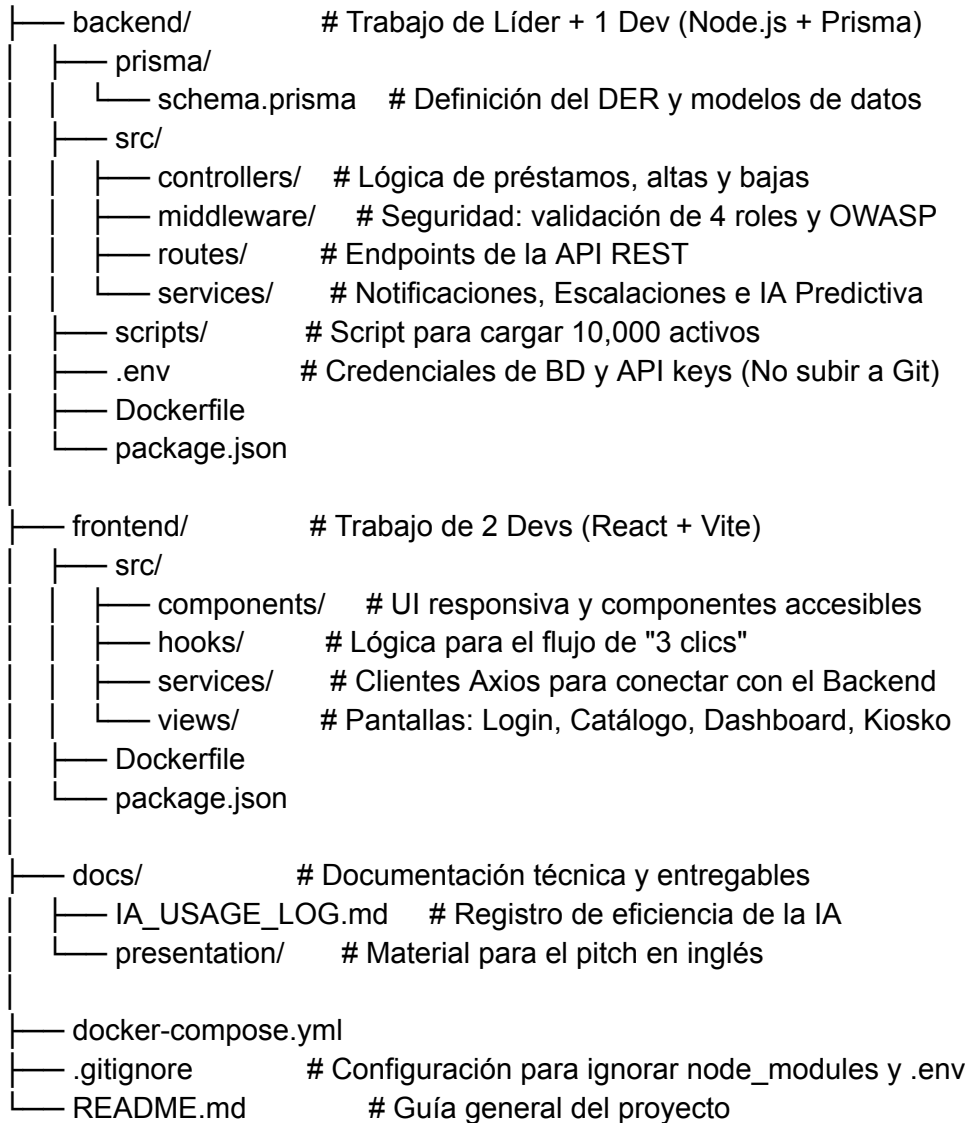
- **Comunicación:** El Frontend pedirá datos mediante JSON (puede cambiar el formato).
- **Seguridad:** Implementaremos protección de *endpoints* basada en los **4 roles** (Usuario, Gerente, Administrador, Auditor), asegurando que nadie acceda a funciones que no le corresponden.

Arquitectura de Trazabilidad

Cada movimiento de un activo (Salida, Devolución, Mantenimiento) generará un registro inmutable en la base de datos para garantizar la **Trazabilidad Total** exigida.

Arquitectura de carpetas

zf-halo-imu/



Base de Datos (Supabase)

Base de Datos: PostgreSQL (alojada y gestionada en **Supabase**).

¿Por qué nos conviene?

1. **Todos vemos lo mismo:** Al estar en la nube, si yo agrego un activo, ustedes lo ven al instante. Se acabó el "pásame tu dump de la base de datos".
2. **Panel de Control (GUI):** Supabase tiene una interfaz web increíble. Pueden ver y editar las tablas como si fuera un Excel, sin pelearse con pgAdmin o comandos raros.

3. **Connection Pooling:** Supabase gestiona las conexiones de Prisma automáticamente. Esto es vital para que la app no se caiga cuando le metamos los **10,000 activos**.

¿Qué tienen que hacer ustedes? Solo asegúrense de tener la variable `DATABASE_URL` en su archivo `.env` (yo se las paso por privado).

Comandos útiles:

- `npx prisma db push`: Sube los cambios de nuestro `schema.prisma` a la nube.
- `npx prisma studio`: Abre un panel local para ver los datos si no quieren entrar a Supabase.

Reglas de Oro para el Desarrollo

Para alcanzar el 100% de la competencia, todos debemos seguir estos principios:

1. **Regla de los 3 Clicks:** Cualquier flujo de solicitud debe completarse en máximo 3 interacciones.
2. **Rendimiento Crítico:** Las consultas a la BD de 10k activos deben resolverse en menos de 2 segundos.
3. **Accesibilidad:** Uso de contrastes adecuados y etiquetas legibles para cumplir con los estándares de accesibilidad.
4. **Innovación (IA):** El backend integrará un módulo de Análisis Predictivo para sugerir demanda de activos basada en el historial.

Docker

Para cumplir con la **Mantenibilidad** y asegurar que el entorno sea idéntico para los 5 integrantes del equipo, utilizamos Docker.

¿Cómo funciona?

Docker empaqueta el Frontend, el Backend y la Base de Datos en "**contenedores**" aislados. Imagina que son "cajas" que ya traen instaladas las versiones exactas de Node.js (LTS) y PostgreSQL que necesitamos.

- **Beneficio:** Eliminamos el error "*En mi máquina sí funciona*". Si corre en Docker, corre igual para todos y para los jueces.
- **Persistencia:** La base de datos guarda la información en un "volumen" virtual, así que tus 10,000 activos no se borran al apagar la computadora.

Guía Rápida de Comandos (Cheatsheet):

Acción	Comando (en la raíz del proyecto)
Encender Todo	<code>docker-compose up --build</code> (Descarga, construye y levanta el sistema)
Apagar Todo	<code>docker-compose down</code> (Detiene y limpia los contenedores)
Verificar Logs	<code>docker-compose logs -f</code> (Muestra errores en tiempo real)
Correr Comandos Backend	<code>docker-compose exec backend <comando></code> (Ej. para Prisma o Scripts)

Servicios Activos:

1. **zf_web (Frontend)**: Accesible en <http://localhost:5173>
2. **zf_api (Backend)**: Accesible en <http://localhost:3000>
3. **zf_db (Base de Datos)**: PostgreSQL corriendo en puerto [5432](#)

Git Workflow

- Main: Solo código estable y probado.
- Dev: Rama de integración.
- Feature/nombre-tarea: Ramas individuales para cada funcionalidad de Trello (deben ser based en Dev).

Mandamientos de Git

Para evitar conflictos y saber qué está pasando en el proyecto, **todos** usaremos el estándar **Conventional Commits**.

1. La Estructura Obligatoria

Todo mensaje de commit debe seguir esta fórmula exacta:

- TIPO: descripción corta y clara

2. Los "TIPOS" permitidos

Solo usaremos estas palabras clave al inicio (siempre en minúsculas y en inglés, es el estándar):

- **feat: (Feature)** Cuando agregas una funcionalidad nueva (ej. "feat: agrega botón de login").
- **fix: (Fix)** Cuando corriges un error o bug (ej. "fix: corrige error al cargar imagen").
- **docs: (Documentation)** Cambios solo en el README o comentarios (ej. "docs: actualiza instrucciones de instalación").
- **style: (Style)** Cambios de formato (espacios, puntos y comas) que no afectan el código.
- **refactor: (Refactor)** Mejoras el código sin cambiar su funcionalidad (limpieza).
- **chore: (Chore)** Tareas de mantenimiento (ej. actualizar librerías, configurar Docker).

3. Reglas de Oro para la Descripción

1. **Imperativo Presente:** Escribe como si le dieras una orden a la computadora.
 -  BIEN: **agrega validación**
 -  MAL: **agregué validación, agregando validación**
2. **Sé Específico:** Di *qué* cambiaste, no *por qué*.
 -  BIEN: **fix: corrige desbordamiento en navbar móvil**
 -  MAL: **fix: arregla el diseño**
3. **Un cambio = Un commit:** No mezcles cosas. Si arreglaste el login y cambiaste el color del footer, **haz dos commits separados**.

4. Ejemplos (Hall of Fame vs. Hall of Shame)

❌ Jamás hagas esto (Cárcel)	✅ Haz esto (Profesional)
cambios	style: ajusta márgenes en tarjeta de activo
arreglo bug	fix: resuelve crash al escanear QR
subiendo cosas	feat: integra endpoint de usuarios
final final v2	chore: prepara build para producción
wip (Work in Progress)	feat: estructura inicial del dashboard (incompleto)

¿Hago Commit o Espero?

El objetivo es guardar tu progreso sin romperle el código a los demás. Sigue estas reglas simples:

● LUZ VERDE: ¡Haz Commit Ahora!

1. **"La regla del videojuego" (Save Point):** Acabas de lograr que algo funcione. Guarda. Si sigues avanzando y lo rompes, podrás volver a este punto seguro.
2. **Terminaste una Tarea Lógica (Atomicidad):** Si la tarea era "*Crear el Navbar*", y ya está creado y se ve bien → **Commit**. No esperes a terminar también el *Footer* para guardar el *Navbar*.
3. **Te vas a ir (Lunch/Dormir):** Nunca dejes código sin guardar al final del día. Si está a medias, haz un commit con el prefijo **wip**: (Work In Progress) para que sepamos que no está listo, pero está a salvo en la nube.
4. **Antes de hacer git pull:** Si necesitas bajar cambios de tus compañeros, primero asegura los tuyos.

● LUZ ROJA: ¡ESPERA! (No hagas Commit)

1. **El código no compila (Rompe el Build):** Si tu código tiene un error que hace que la pantalla se ponga blanca o el servidor se caiga, **PROHIBIDO** subirlo a la rama `main` o `dev`. Arréglalo primero. Bloquearás a todo el equipo si lo subes roto.
2. **Archivos Basura:** Revisa que no estés subiendo `node_modules`, archivos `.env` con contraseñas, o logs de error (`npm-debug.log`).
3. **Hiciste 3 cosas a la vez:** Arreglaste el login, cambiaste el color del fondo y moviste la base de datos... **¡Alto!** No hagas un solo commit gigante.
 - *Solución:* Usa `git add` por partes y haz 3 commits pequeños.

● LUZ AMARILLA: Ten Cuidado

1. **"Solo estoy probando algo":** Si estás experimentando y no sabes si va a funcionar, es mejor no hacer commit todavía, o hacerlo en una **rama temporal** (ej. `temp/prueba-animacion`).
2. **Cambios de formato automático:** Si tu editor de código reformateó todo el archivo (cambió espacios por tabs en 500 líneas) pero solo cambiaste una línea de código real.
 - *Tip:* Revisa bien qué estás subiendo. Esos cambios ensucian el historial.

Nota: En general, si tienen dudas mejor pregunten